

The Unreasonable Effectiveness of Events

Mastodon @lutzhuehnken@mastodon.social

LinkedIn <https://linkedin.com/in/lutzh>

Blog <https://www.reactivesystems.eu/>



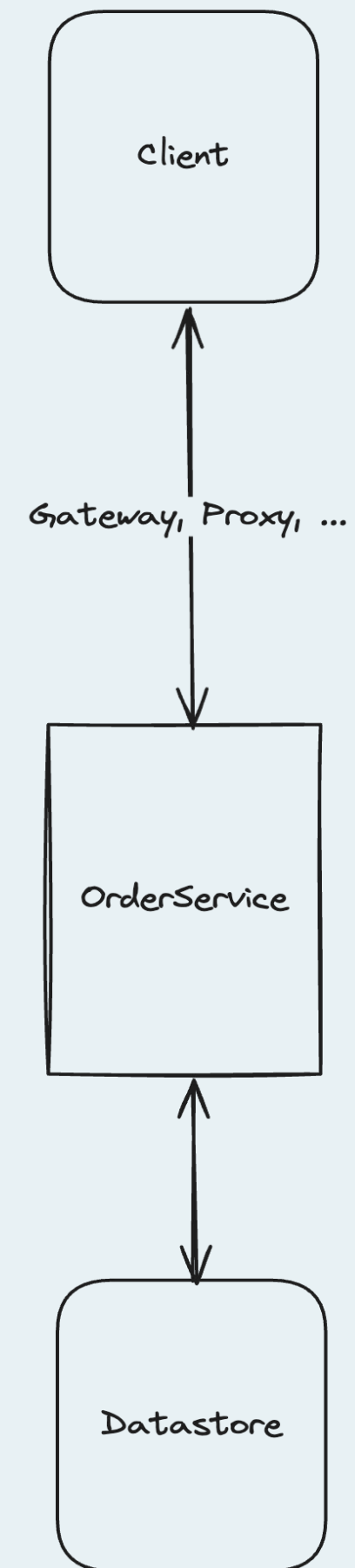
A blue t-shirt is displayed against a light gray background. In the center of the chest, there is a white circle. Inside the circle, the text "I wrote this without ChatGPT" is written in white, sans-serif font, arranged in three lines.

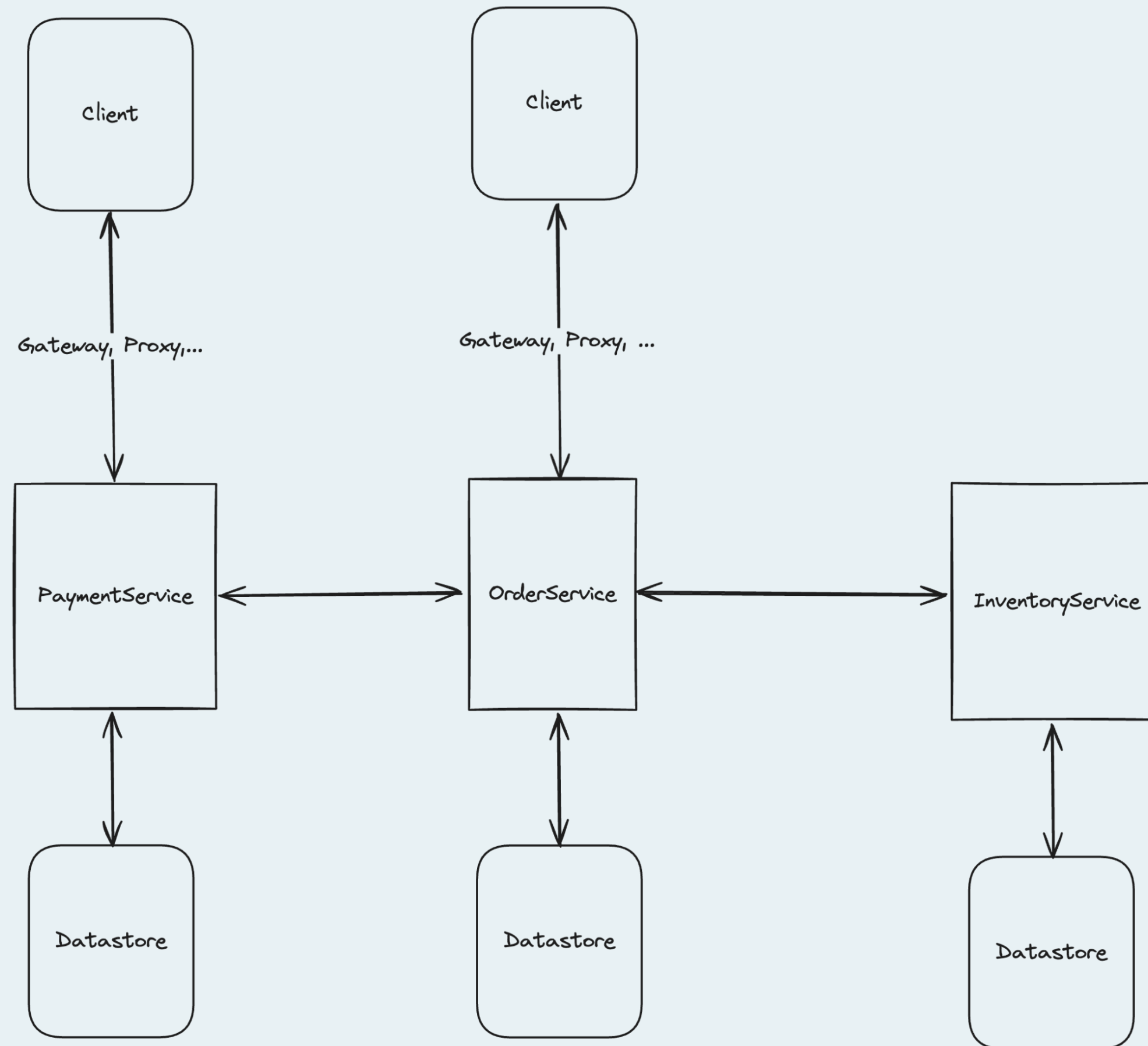
I wrote this
without
ChatGPT

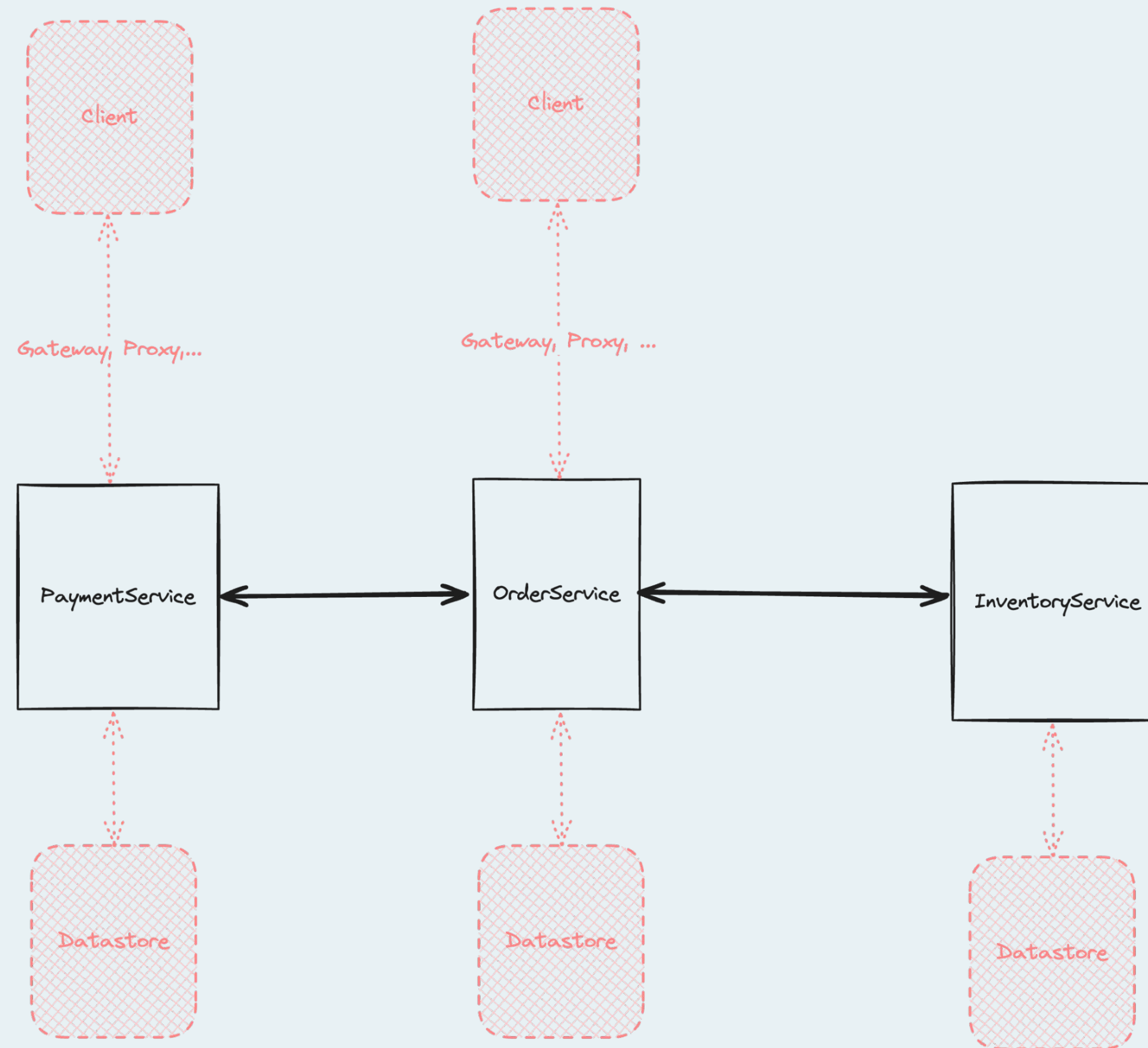


Why me?

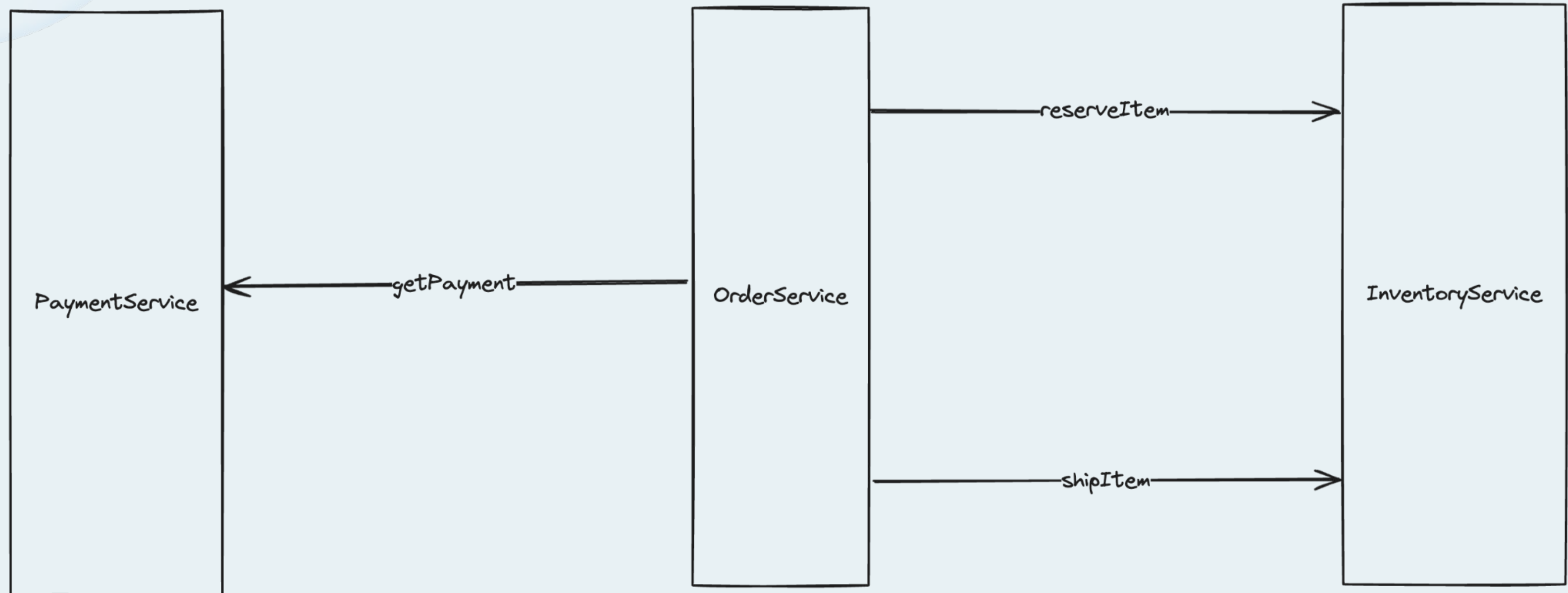
- Built my first message-driven, asynchronous system for the Bundesbank in 2002
- Worked on various Event-Driven systems, e.g. for Zalando, ING, ista, Maersk
- Currently building a bank (!) with Event-Driven Architecture, Microservices, DDD



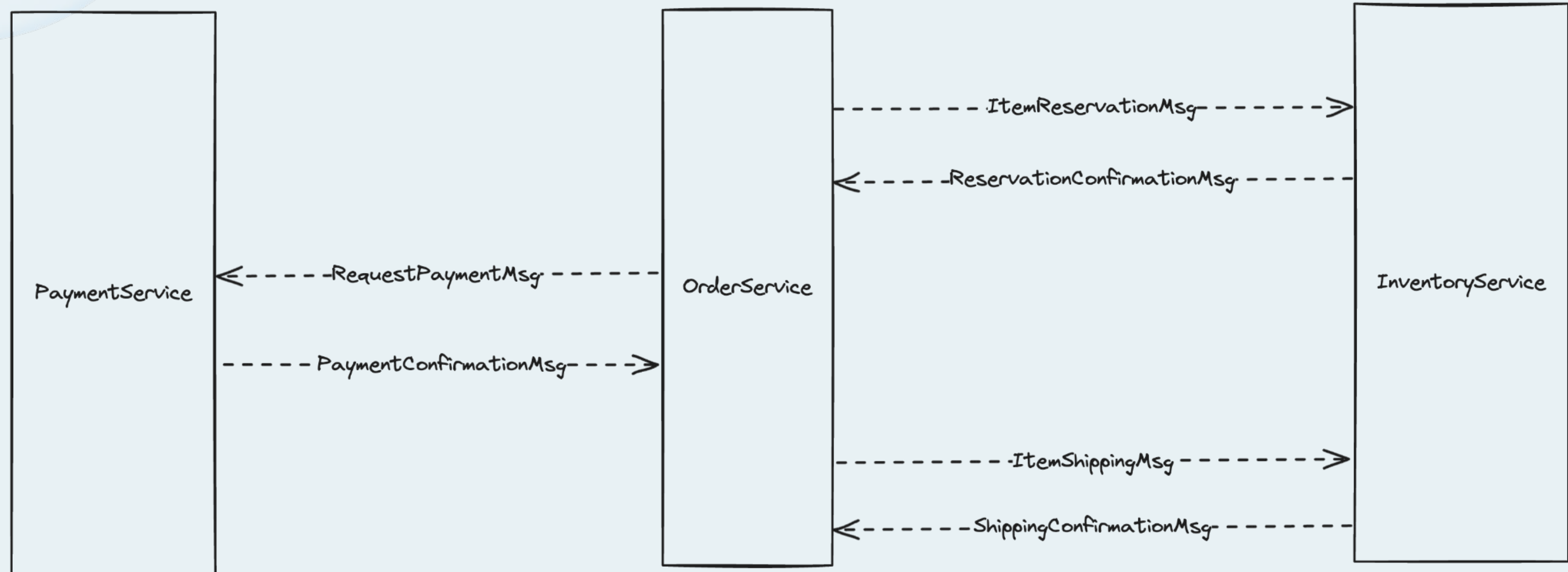




Synchronous calls (e.g. HTTP, gRPC)



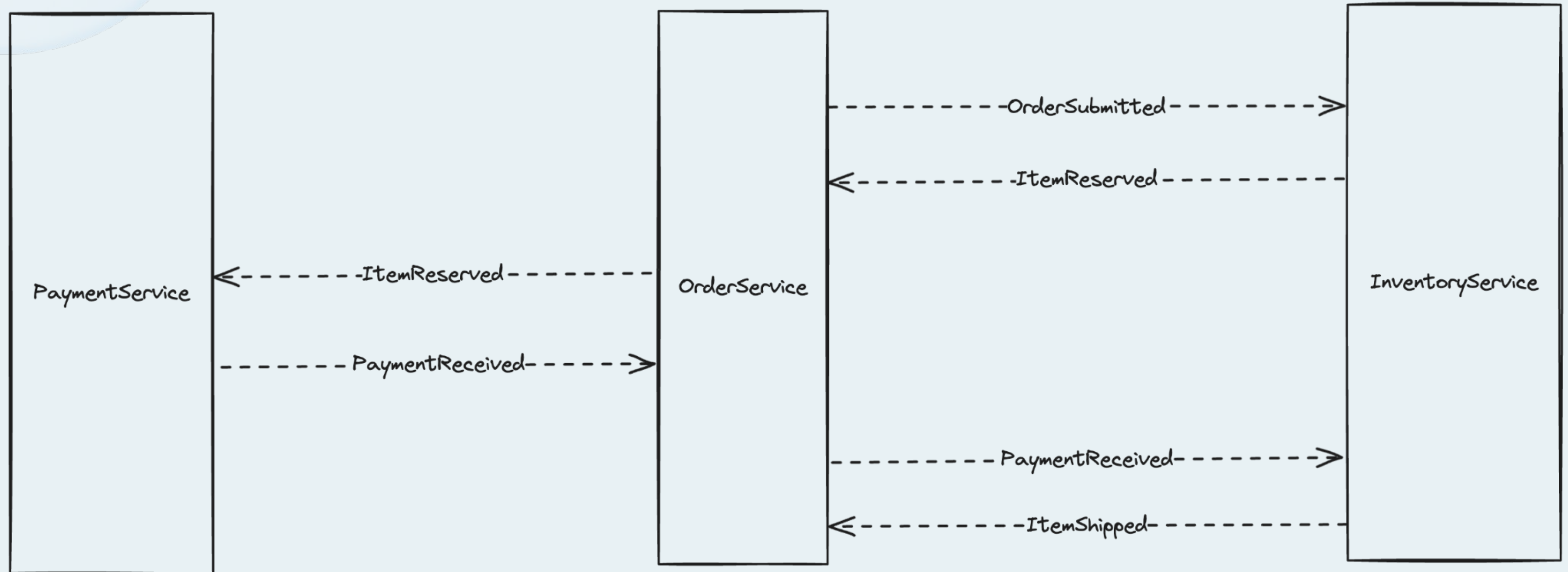
Asynchronous messaging

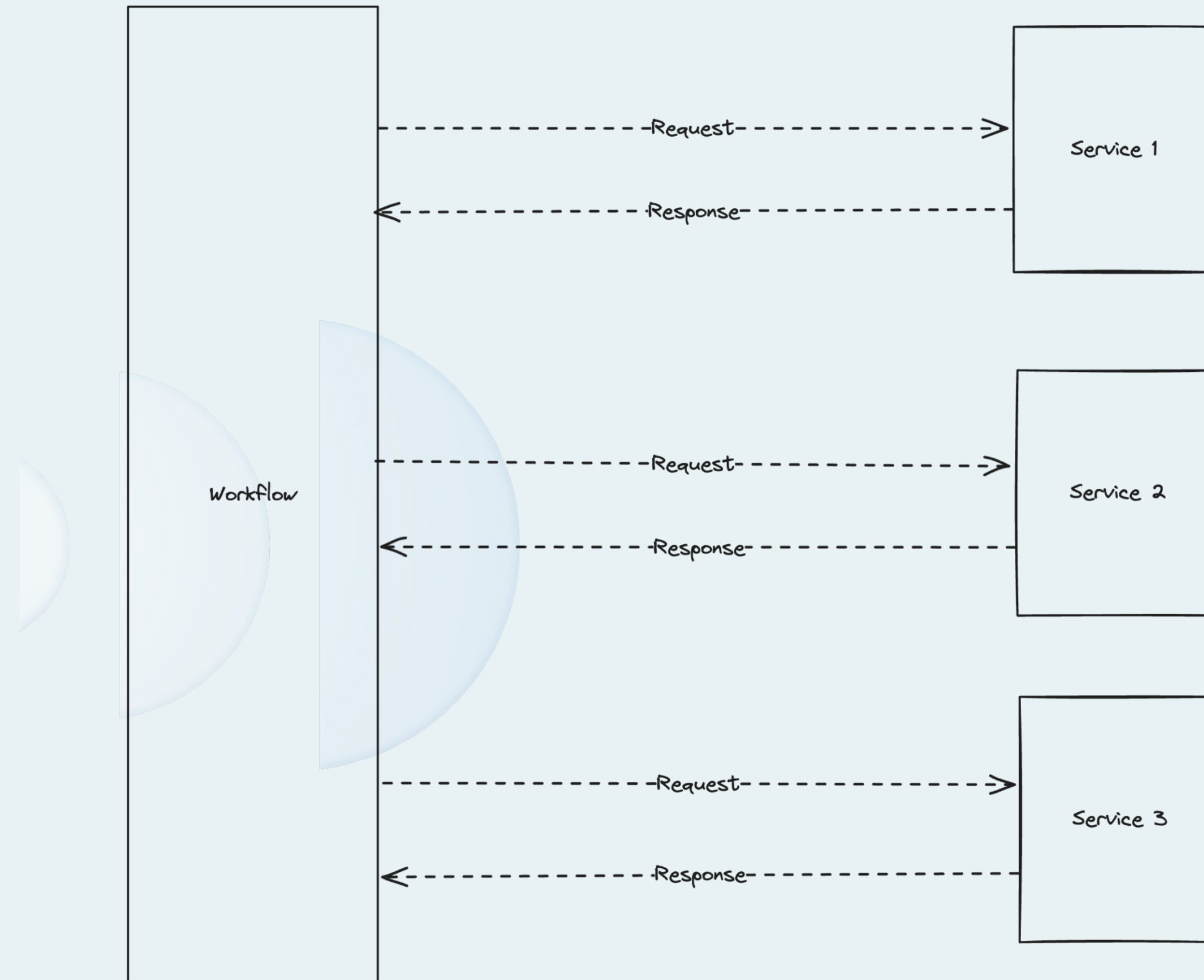


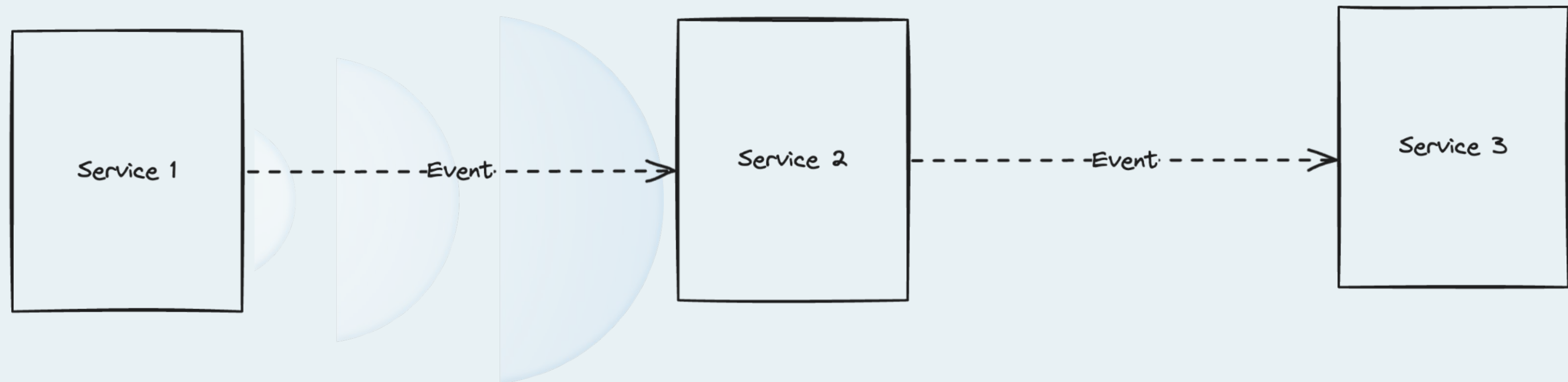
Reminder: Queries, Commands, Events

	Is..		
Query	A request for information about the current state of one or many objects		
Command	An intention to perform an operation or change a state		
Event	A fact, something that undisputedly happened in the past		

Events







Reminder: Queries, Commands, Events

	Is..	Expected Response	
Query	A request for information about the current state of one or many objects	The requested information	
Command	An intention to perform an operation or change a state	A confirmation that the command has been executed, or an error message if the command failed	
Event	A fact, something that undisputedly happened in the past	None (events are facts, they can't "fail")	

Reminder: Queries, Commands, Events

	Is..	Expected Response	Communication Pattern
Query	A request for information about the current state of one or many objects	The requested information	Request-Response
Command	An intention to perform an operation or change a state	A confirmation that the command has been executed, or an error message if the command failed	Request-Response
Event	A fact, something that undisputedly happened in the past	None (events are facts, they can't "fail")	Fire-and-Forget

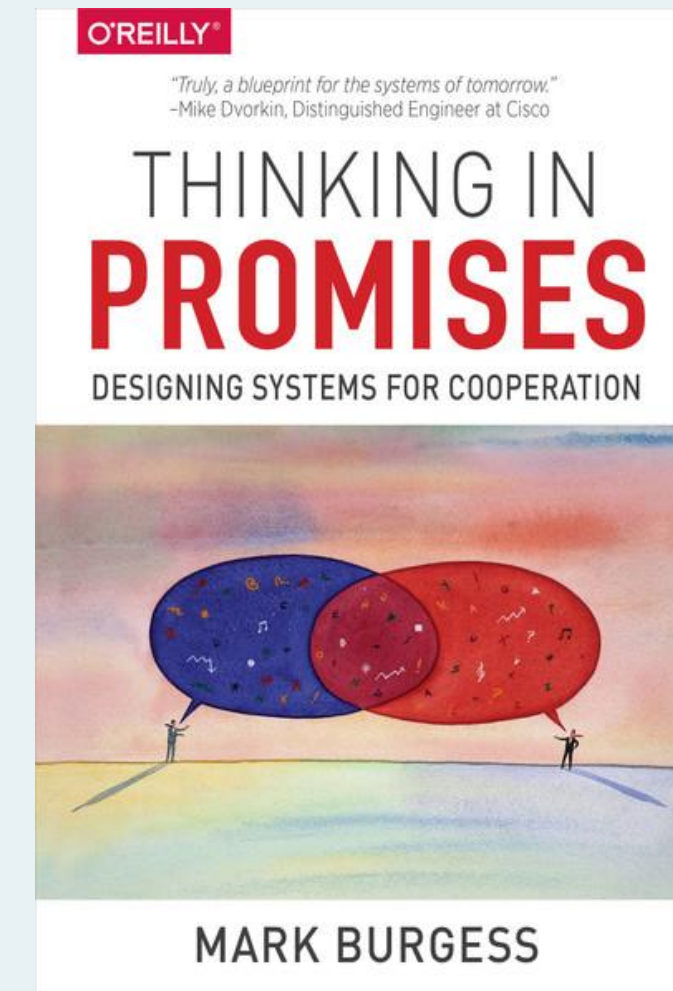
Reminder: Queries, Commands, Events

	Is..	Expected Response	Communication Pattern
Query	A request for information about the current state of one or many objects	The requested information	Request-Response
Command	An intention to perform an operation or change a state	A confirmation that the command has been executed or a message back to the command	Request-Response
Event	A fact, something that has happened in the past	None (events are facts, they can't "fail")	Fire-and-Forget

But how can I know it'll be picked up and processed?

Mental model 1/2

Thinking in Promises



(-b)
A1 → A2

(-OrderCreated)
PaymentService → OrderService

Agents

Agents in Promise Theory are said to be autonomous, meaning that they are causally independent of one another. This independence implies that they cannot be controlled from without, they originate their own behaviours entirely from within, yet they can rely on one another's services through the making of promises to signal cooperation.

Promises

Promises arise when an agent shares one of its intentions with another agent voluntarily, e.g. by publishing its intent.

https://en.wikipedia.org/wiki/Promise_theory

Synchronous Command	Asynchronous Command	Asynchronous Event
		

**Events are
“fire-and-forget”...**

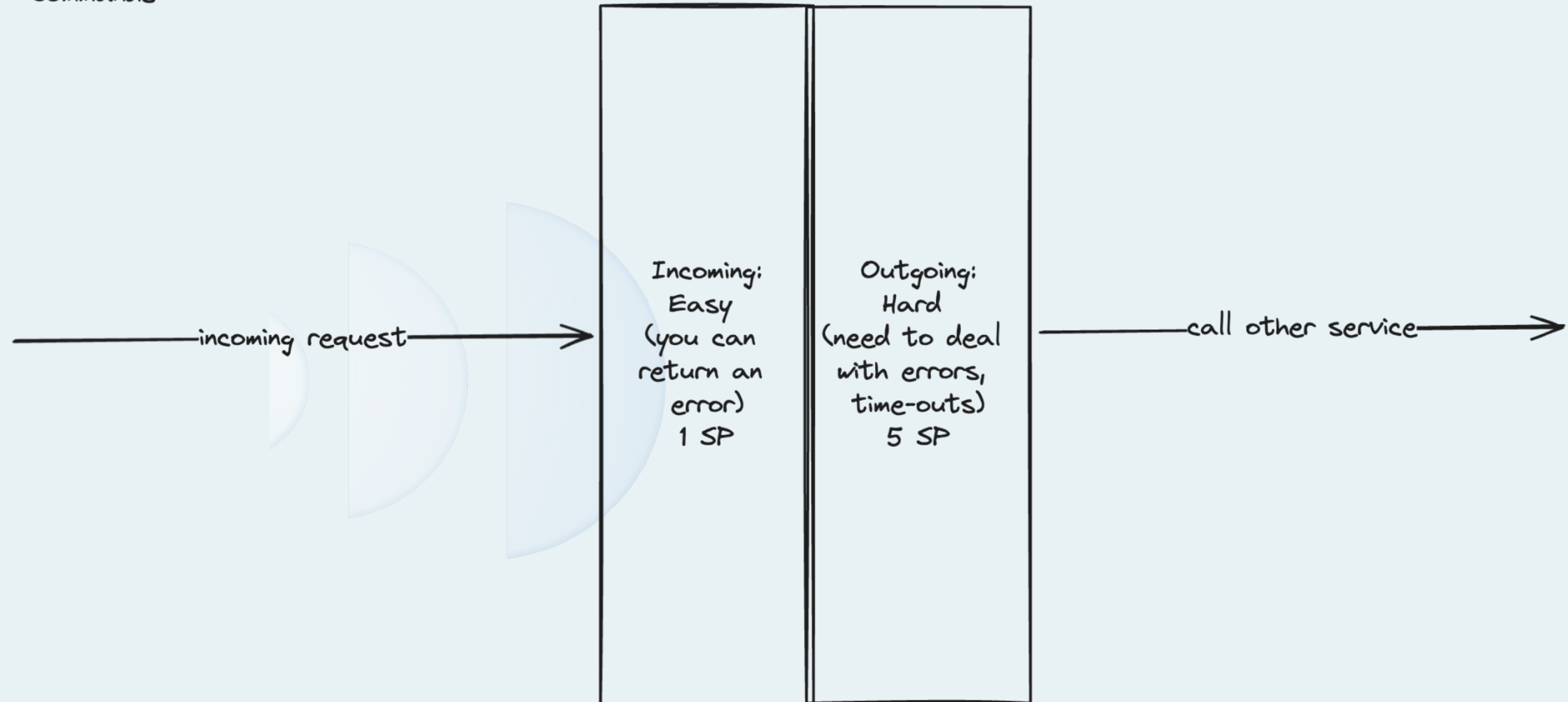
**... but based on previous
agreements (promises)**

Reminder: Queries, Commands, Events

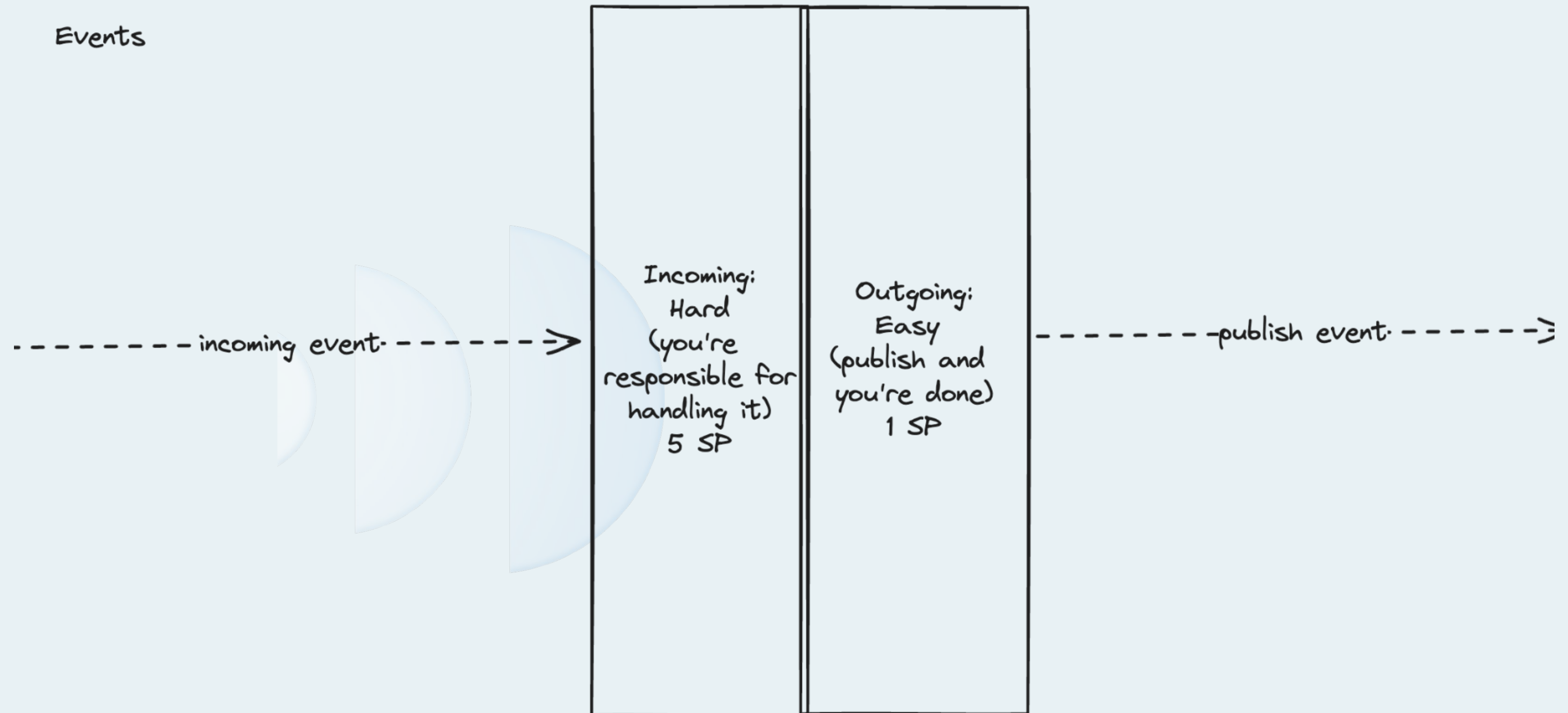
	Is..	Expected Response	Communication Pattern
Query	A request for information about the current state of one or many objects	The requested information	Request-Response
Command	An intention to perform an operation or change a state	A confirmation, a success message, or an error message if the command failed	Request-Response
Event	A fact that has already happened in the past	None (events are facts, they can't "fail")	Fire-and-Forget (rely on promises)

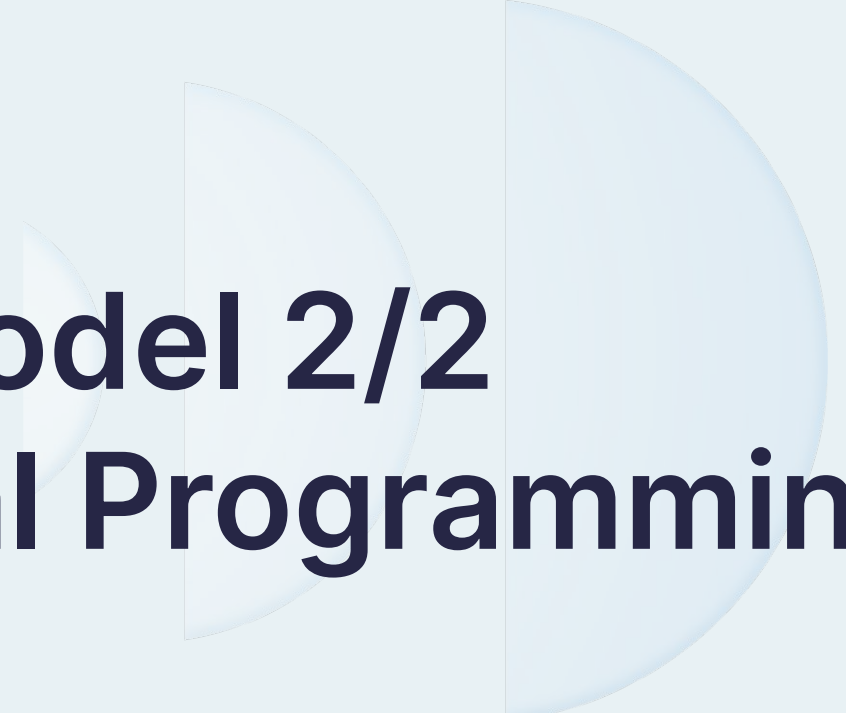
But surely there'll still be errors we need to handle?

Commands



Events

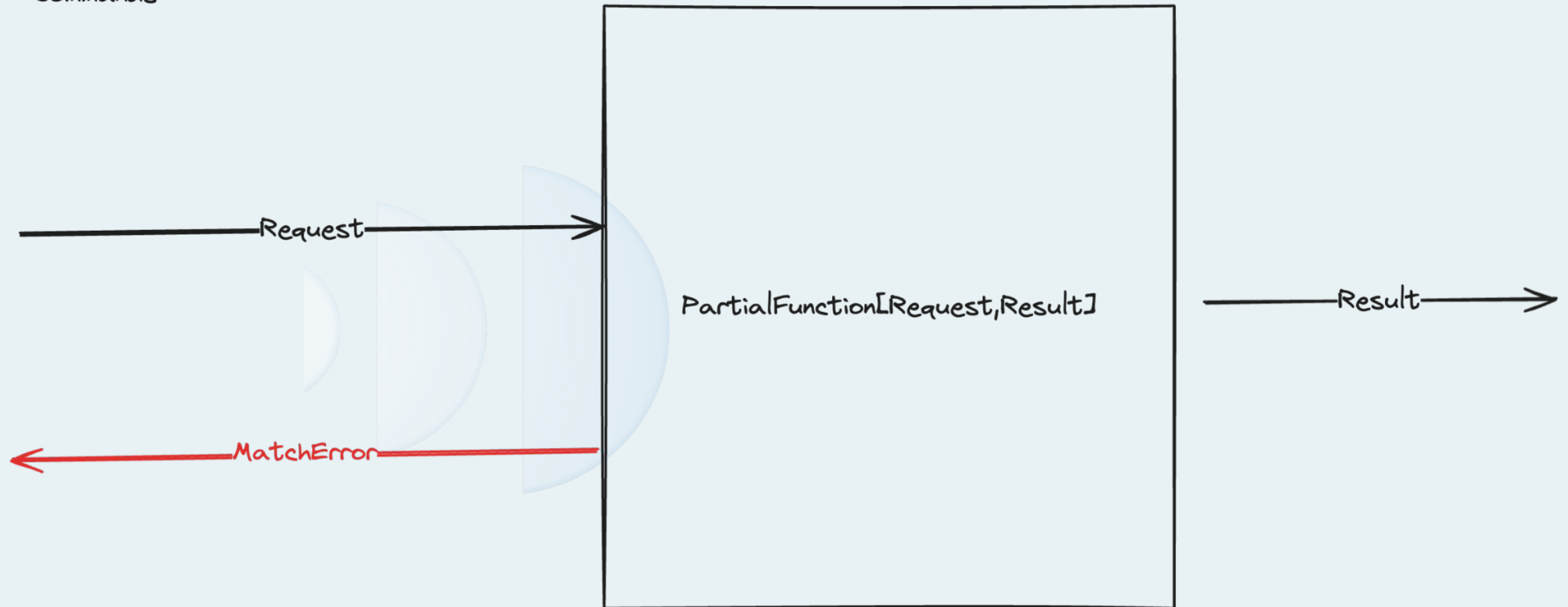




Mental model 2/2

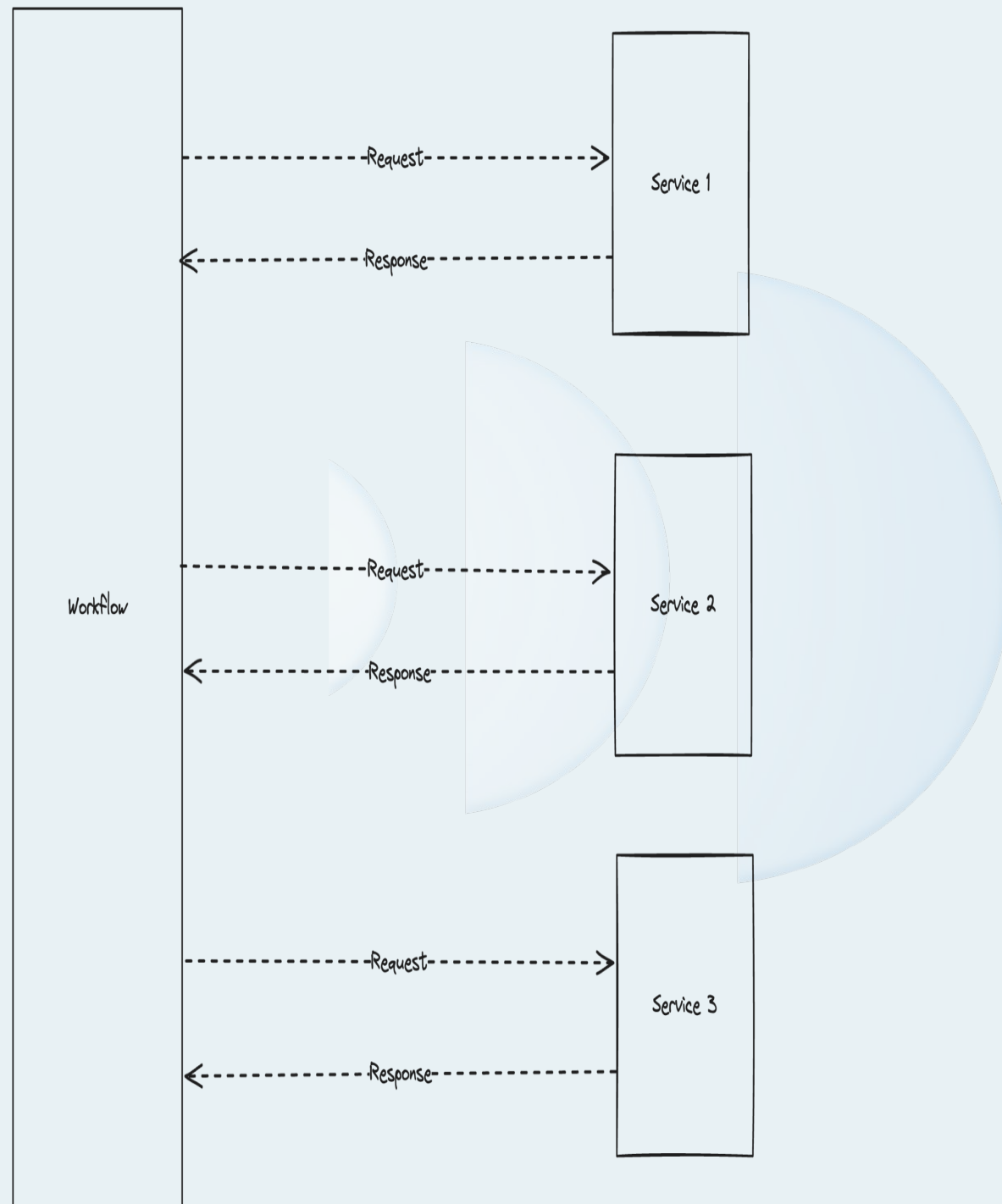
Functional Programming

Commands



Events

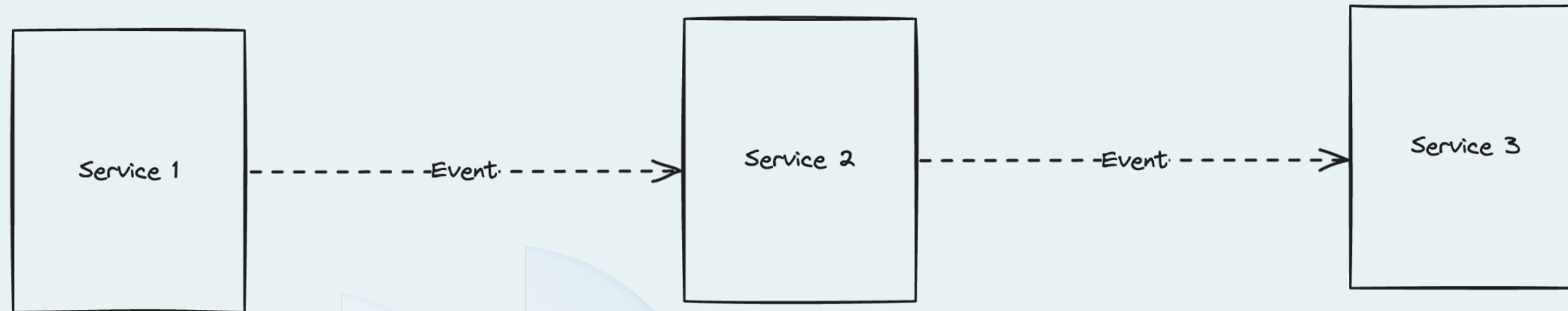




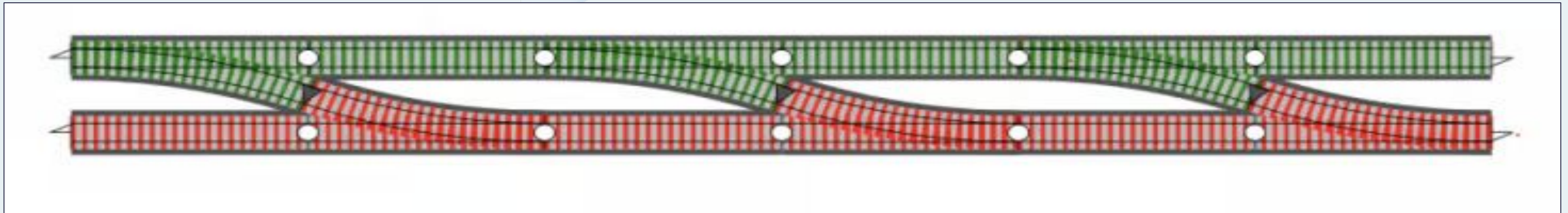
```
item, err := reserveItem(itemNumber)
if err != nil {
    return nil, errors.New("out of stock")
}

confirmation, err := getPayment(itemNumber)
if err != nil {
    return nil, errors.New("insufficient funds")
}

result, err := shipItem(itemNumber)
```



```
reserveItem(itemNumber).flatMap(getPayment).flatMap(shipItem)
```



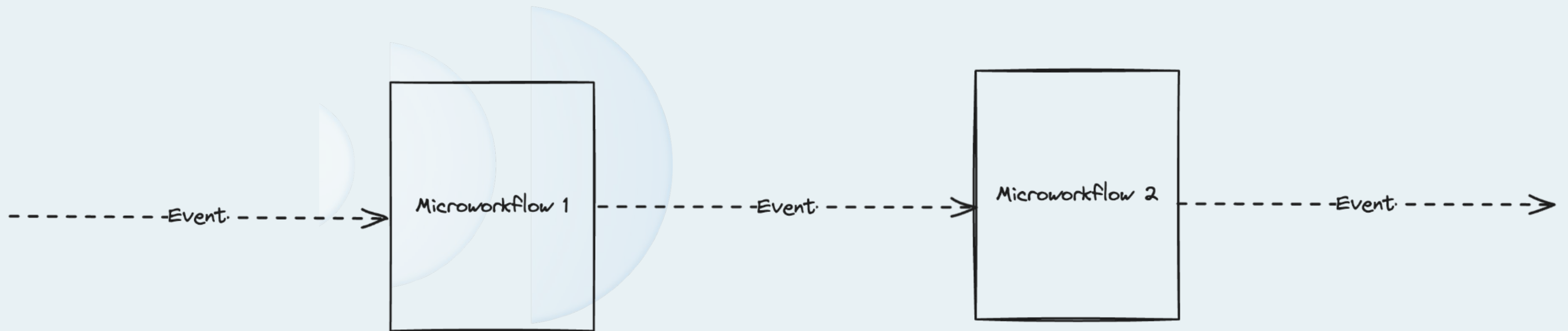
**The unreasonable effectiveness of events:
Beyond asynchronous communication, being
event-driven has a massive effect on the
overall workflow and system design.
Event-Driven Architecture (EDA) is at least as
much about the flow as it is about the actual
events.**

Mindbender:

We could treat a command like an event

	Is..	Expected Response	Communication Pattern
Command	An intention to perform an operation or change a state	None (receiver is not allowed to reject)	Fire-and-Forget
Event	A fact, something that undisputedly happened in the past	None (events are facts, they can't "fail")	Fire-and-Forget

Maybe, instead of Event-Driven Architecture, we should talk about microworkflows?



Coming soon: <https://microworkflows.org>

The unreasonable effectiveness of events:

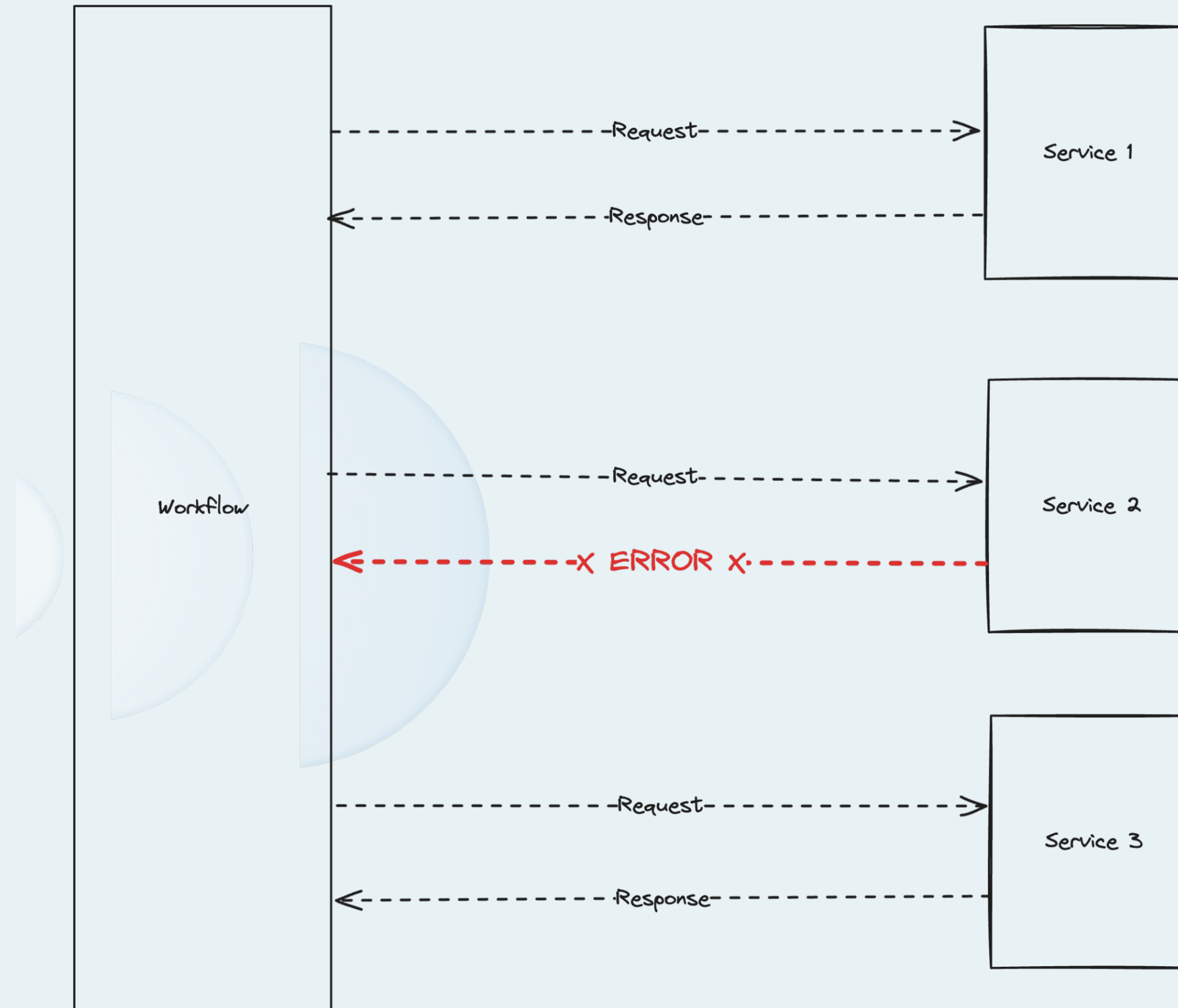
- scalability and resilience
- testability
- organizational clarity / domain boundaries

Enjoy the (load) testing

Fewer (no?) runtime dependencies to other services = simpler tests.

Testing an event-driven service (or a microworkflow) is still an integration test.

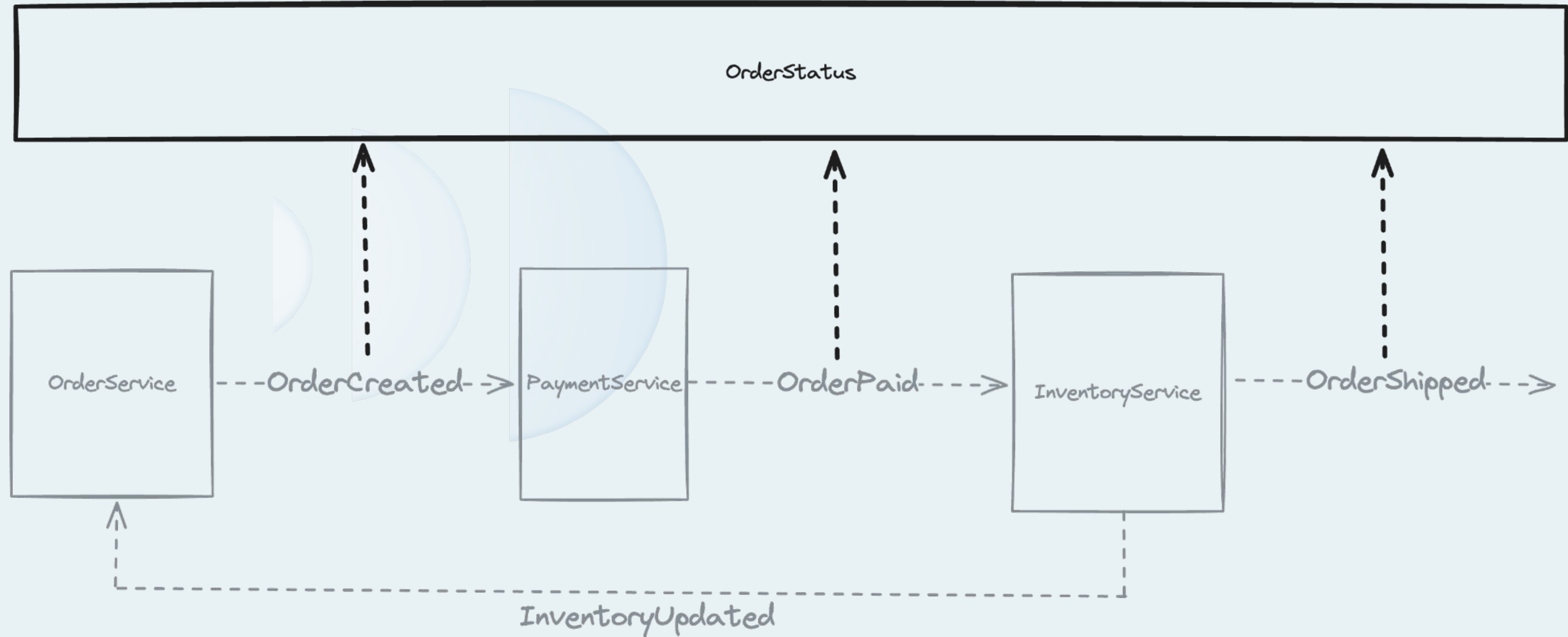
But it's nice to be able to test e.g. throughput in a very isolated way.



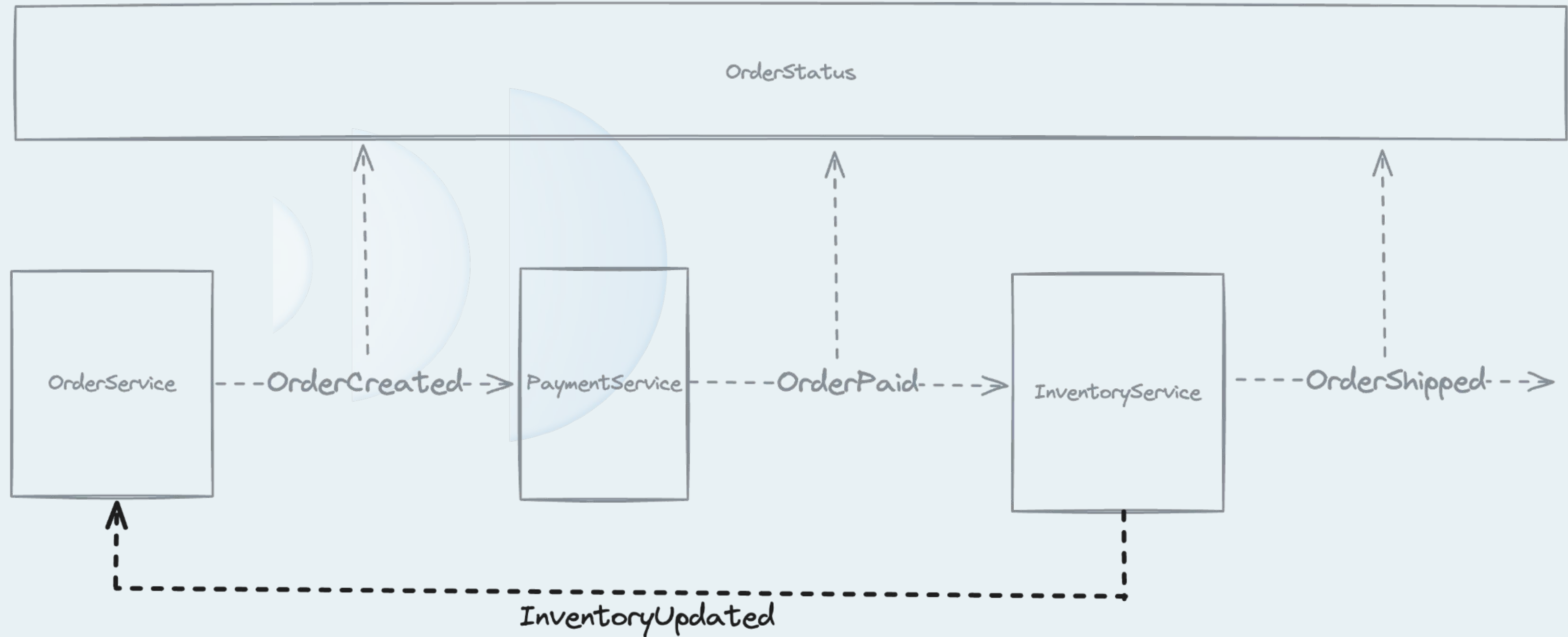
Important Patterns

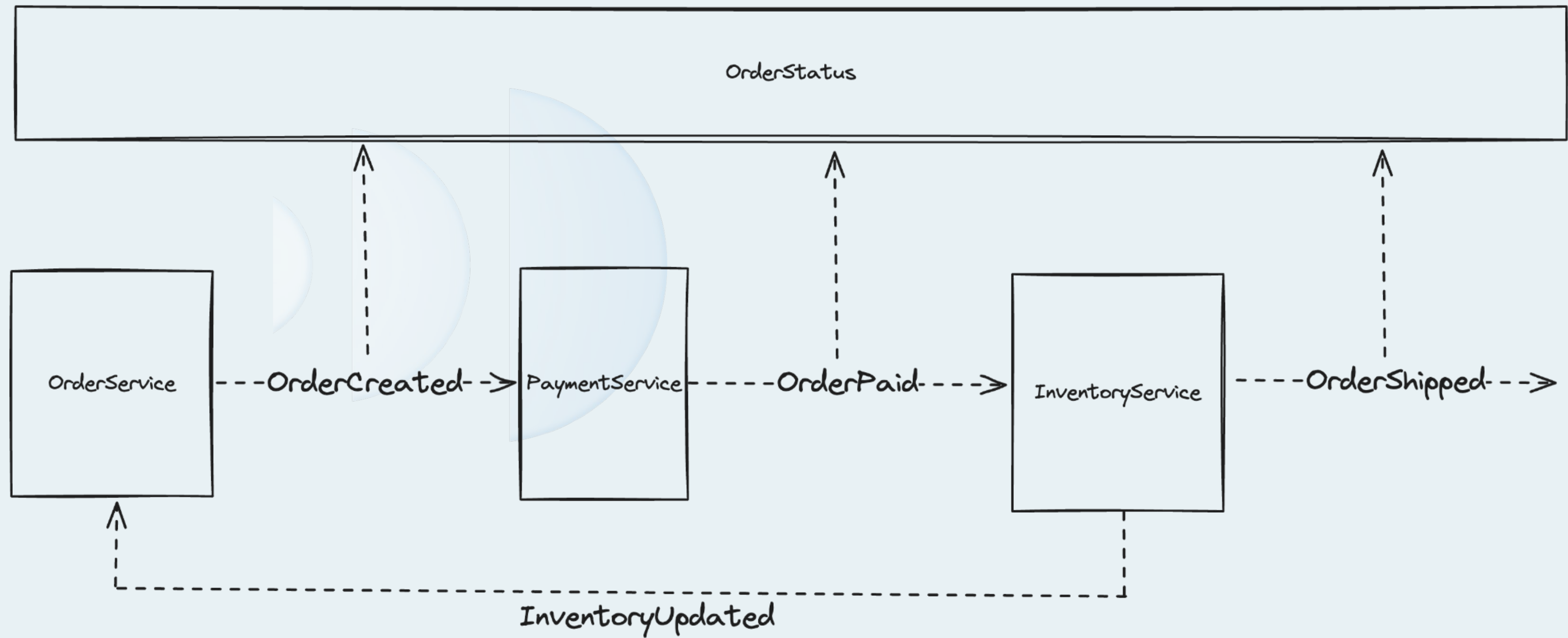


Separate Observing from Control



Bring the data to the process





Bonus models

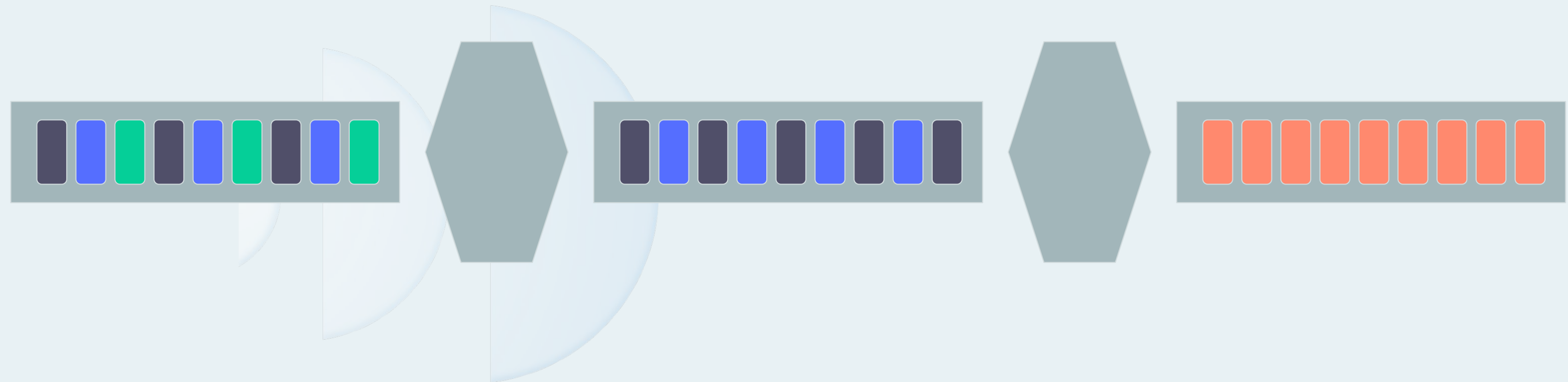


Think Unix Philosophy

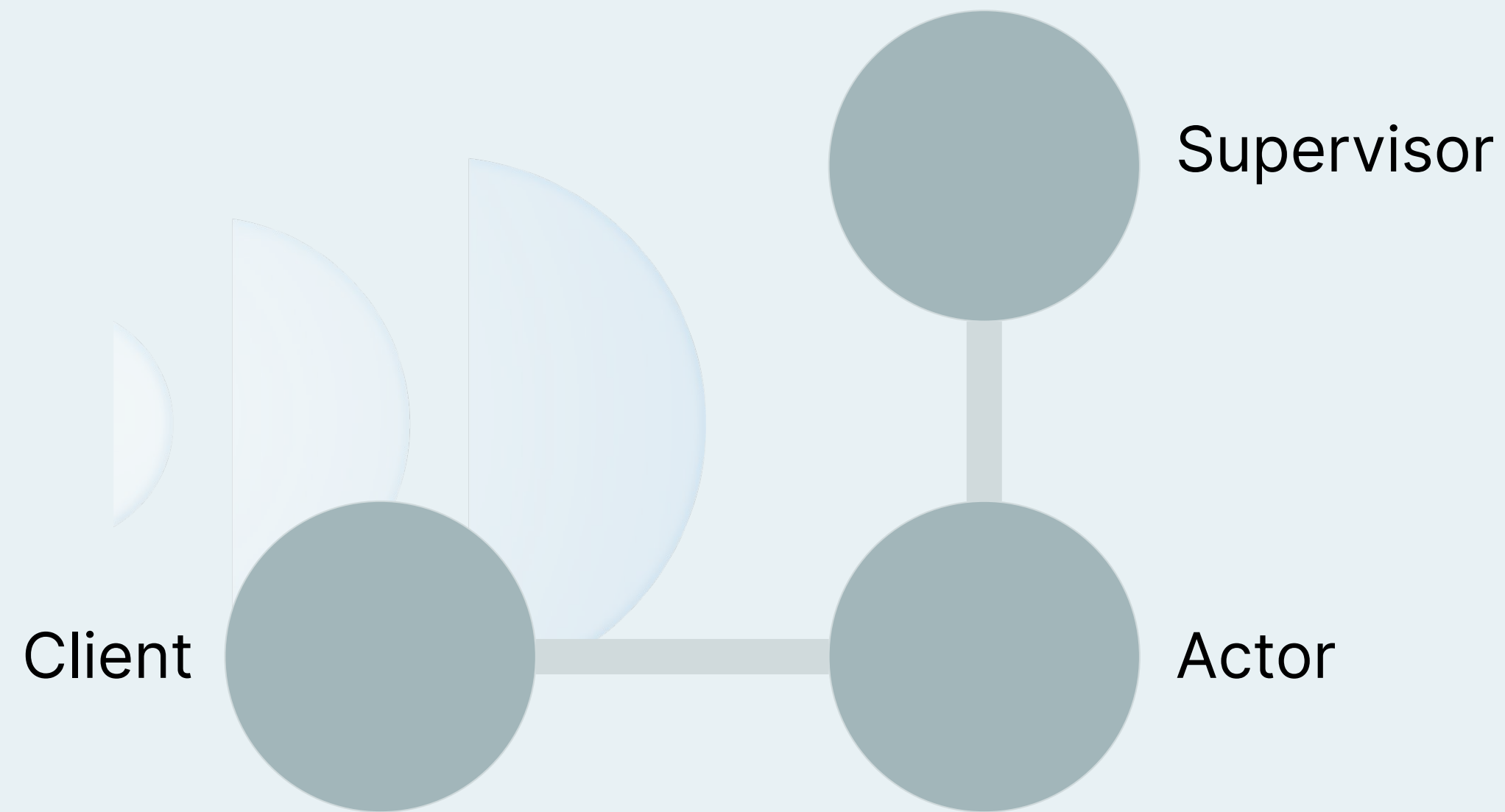
1. Make each program do one thing well. To do a new job, build afresh rather than complicate old programs by adding new "features".
2. Expect the output of every program to become the input to another, as yet unknown, program. ...

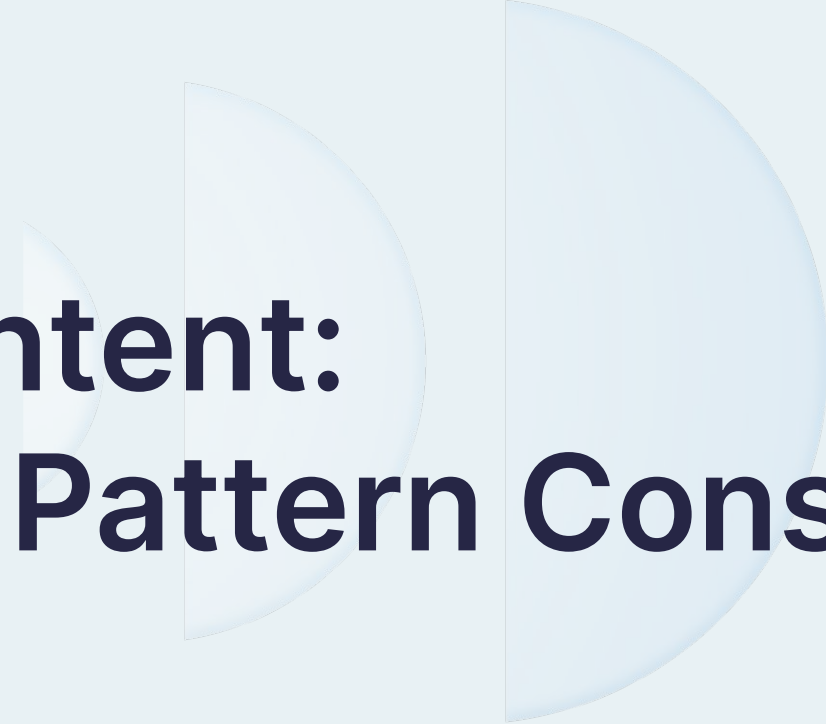
```
$ Cat file3.txt | grep "dwx" | tee file4.txt | wc -l
```


Think Stream Processing

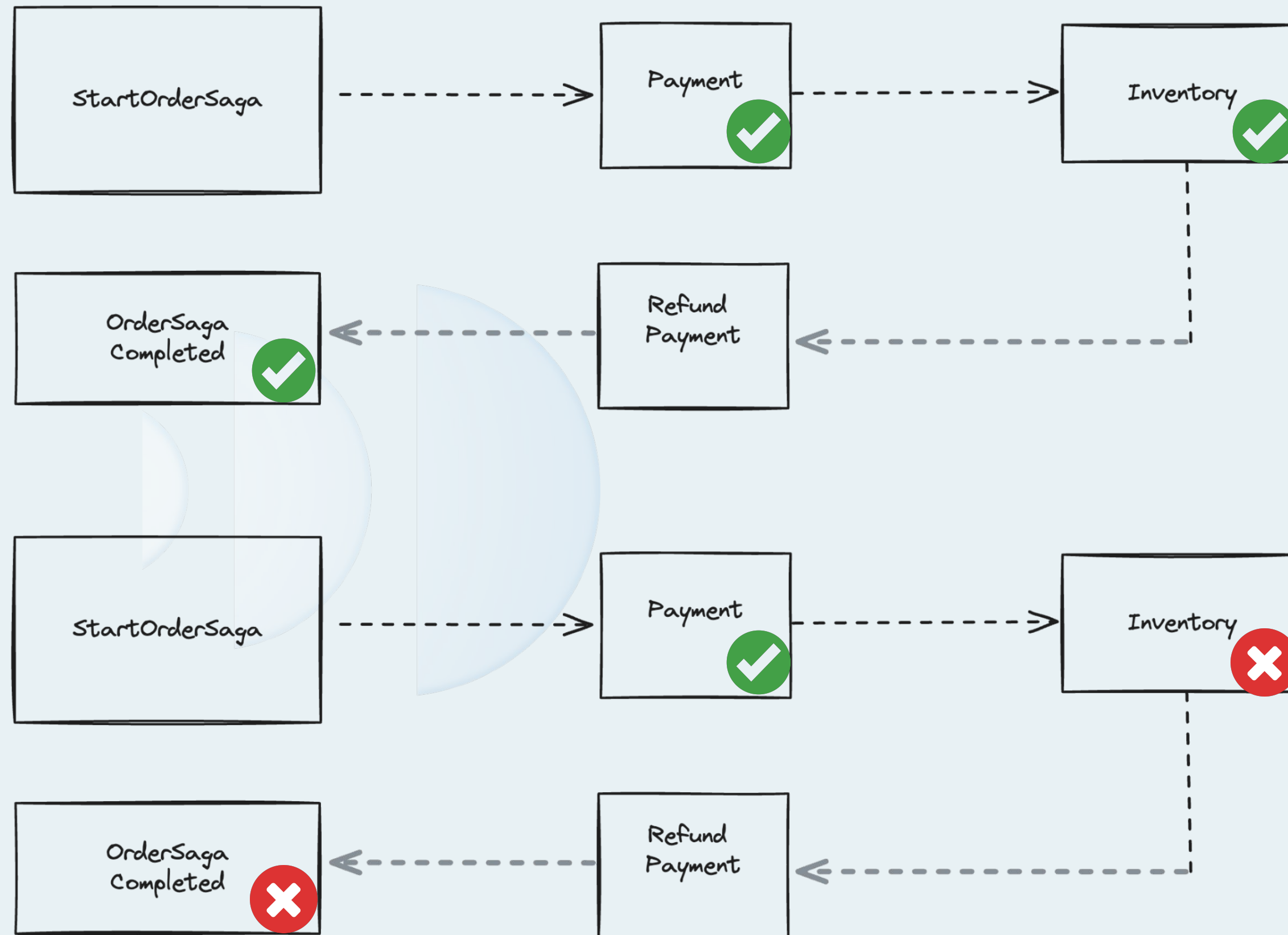


Think Actors





Bonus content: The Saga Pattern Considered Harmful



Event-Driven Architecture

enables scalability, resilience

promotes clear responsibility boundaries

Event-Driven Architecture

Thinking in events is not enough - you'll also need to think in terms of promises and railways.

Make sure not to approach it with an imperative mindset.

What do you think? Let's discuss!

Please rate your experience ★★★★★

Mastodon @lutzhuehnken@mastodon.social

LinkedIn <https://linkedin.com/in/lutzh>

Blog <https://www.reactivesystems.eu/>

